

`sale:repriceAll([undercut=1]) → number`

Bloquant

Perm.Hdv

Comme UpdateAllItems avec une sous-cote au choix : chaque annonce passe au prix min du marché – undercut (garde anti-auto-sous-cote). Renvoie le nombre repositionné.

```
sale:repriceAll(5)
```

Copier

`sale:refreshPrices()`

Bloquant

Perm.Hdv

Lecture seule : rafraîchit le prix marché de chaque GID que tu as en vente (ancien comportement de UpdateAllItems).

`sale:ItemsOnSale() → number`

Nombre d'annonces que tu as en vente (emplacements HDV occupés).

`sale:GetItemGID(index) → number`

`sale:GetItemGUID(index) → number`

`sale:GetItemPrice(index) → number`

`sale:GetItemQuantity(index) → number`

Pour ton annonce n° index (1-based comme SnowBot, triées par GID puis quantité ; 0 = alias de 1) : son GID, son GUID (uid d'échange), son prix demandé, sa quantité.

```
for i = 1, sale:ItemsOnSale() do
    print(getItemName(sale:GetItemGID(i)) .. " @ " .. sale:GetItemPrice(i))
end
```

Copier

`sale:getGID(GUID) → number`

GID d'une de tes annonces à partir de son GUID.

`sale:GetLots(quantité) → number`

Index SnowBot du lot pour une quantité : 1→1, 10→2, 100→3, 1000→4 ; 0 si la quantité n'est pas un lot standard.

`sale:AvailableSpace()` → `number`

Emplacements HDV libres = nombre max d'annonces du compte (conditions de vente reçues à l'ouverture du HDV vendeur) – tes annonces en cours ; -1 tant qu'aucun HDV vendeur n'a été ouvert depuis la connexion.

`sale:buyItem(GID, quantité, prixMax)` → `bool`

Bloquant

Perm.Hdv

Mode ACHAT (`goToHdvBuy`) : recherche l'offre du GID et achète UN lot de la quantité demandée (1/10/100/1000). `prixMax` = plafond du prix du lot (0 = sans plafond) ; false si le lot n'est pas coté ou trop cher.

```
if goToHdvBuy(HDV_MAP_ID) then
    sale:buyItem(289, 100, 25000)    -- lot de 100 Blé, 25 000 k max
end
```

Copier

`sale:buyToPnj(gid, quantité)` → `false`
`sale:sellToPnj(gid, quantité)` → `false`
`sale:sellAllItemToPnj()` → `nil`
`sale:sellAllItemToPnjSecret()` → `nil`

Boutique PNJ (SnowBot) : REFUSÉ tant que le message de jeu n'est pas capturé — aucun paquet n'est deviné. Avertissement (une fois) ; `buyToPnj` / `sellToPnj` renvoient false, `sellAllItemToPnj*` nil. Le champ `sale.itemToUnfollow` vaut 0.

```
-- Vente : état de mes annonces puis reprise automatique
if goToHdv(HDV_MAP_ID) then
    for i = 1, sale:ItemsOnSale() do
        local gid, qty = sale:GetItemGID(i), sale:GetItemQuantity(i)
        print(getItemName(gid) .. " x" .. qty .. " @ " .. sale:GetItemPrice(i))
    end
    local n = sale:UpdateAllItems()    -- sous-cote de 1 (jamais à la hau
    printSuccess(n .. " annonces repositionnées")
end
```

Copier

```

-- Variante manuelle : sous-coter de 1 chaque annonce plus chère que le marché
for i = 1, sale:ItemsOnSale() do
    local gid, qty = sale:GetItemGID(i), sale:GetItemQuantity(i)
    local minP = sale:GetPriceItem(gid, qty) -- prix min marché du lot
    if minP > 0 and minP < sale:GetItemPrice(i) then
        sale:EditPriceByGID(gid, minP - 1, qty)
    end
end
end

```

Module map: — carte courante

Getters sur la carte courante (monstres, ressources, PNJ, joueurs, zones) et verbes SnowBot (changeMap, door, voyage, zaaps...). Renvoient des listes de tables (indexées 1..n façon Lua). Un argument [mapId] omis = carte courante.

map:**getId()** → number

map:**id()** → number

map:**currentMapId()** → number

Id de la carte courante (0 si inconnu). id() et currentMapId() sont des alias.

map:**currentMap()** → string

map:**currentCell()** → number

Coordonnées "x,y" de la carte (" " si inconnues) / cellule du perso (-1).

map:**onMap**(mapId | "x,y") → bool

map:**onCell**(cell) → bool

true si le perso est sur cette carte (id ou coordonnée) / sur cette cellule.

```
if map:onMap("-15,39") then gatherAll() end
```

Copier

map:**idAt**(x, y) → number

map:**GetMapIdFromCoords**(x, y) → number

map:**getMapIdFromCoords**(x, y) → number

Id de la carte à la coordonnée (x, y) sur le même calque que le perso (0 si inconnu).
GetMapIdFromCoords / getMapIdFromCoords = noms SnowBot.

map:coordsOf([mapId]) → x, y

map:getX([mapId]) → number

map:getY([mapId]) → number

Coordonnées monde d'une carte (nil si inconnues).

```
local x, y = map:coordsOf()
```

Copier

map:pathDistance([from,] to) → number

map:getPathDistance([from,] to) → number

Nombre de sauts À PIED dans le world-graph (zaaps non comptés) : 0 = même carte, -1 = injoignable. Un seul argument = depuis la carte courante.

map:neighbourId(dir [, mapId]) → number

Carte réellement atteinte par changeMap(dir) depuis cette carte (0 si aucune sortie).

map:currentSubAreaId([mapId]) → number

map:currentSubArea([mapId]) → string

map:subAreaName([subAreaId]) → string

map:currentAreaId([mapId]) → number

map:currentArea([mapId]) → string

map:areaName([areaId]) → string

Sous-zone et zone : id ou nom (-1 / "" si inconnus). ⚠ Différences SnowBot (noms mélangés chez lui) : currentAreaId() renvoie ici le VRAI id de zone (SnowBot : l'id de SOUS-zone), currentArea() le NOM de la zone (SnowBot : l'id de zone en chaîne) et currentSubAreaId() sans argument l'id de sous-zone (SnowBot : l'id de ZONE). currentSubArea() rend le nom de la sous-zone des deux côtés. Adaptez les comparaisons d'un script porté. areaName(0) = Amakna est valide.

map:getMonsters() → table[]

Groupes de monstres : { cell, id, name, arch, count, level, monsters = {{gid, name, count}}, members = {{gid, level}} } par entrée. arch = true si le groupe contient un archimonstre ; level = somme des niveaux (0 si inconnu).

```
for _, g in ipairs(map:getMonsters()) do
    print(g.name .. " x" .. g.count .. " niv " .. g.level .. " en " .. g.cell)
end
```

Copier

map:getArchMonsters() → table[]

Seulement les groupes contenant un archimonstre : { cell, id, name (de l'archi), gid } par entrée.

```
local a = map:getArchMonsters()[1]
if a then notify("Archi : " .. a.name) end
```

Copier

map:getNpcs() → table[]

PNJ de la carte : { cell, name, id, npcId, label } par entrée. name = nom localisé SEUL ("PNJ <id>" si le nom est inconnu) ; id = id d'acteur (négatif) ; npcId = id de MODÈLE (celui des scripts SnowBot, 522 = banquier...) ; label = ancien libellé « Nom (npcId) », gardé pour les scripts qui le comparaient. Pour retrouver un PNJ, compare npcId (le nom dépend de la langue du client) : talkNpc et npc:* acceptent l'id de modèle comme l'id d'acteur ; un modèle absent de la carte = PNJ introuvable.

```
for _, p in ipairs(map:getNpcs()) do
    if p.npcId == 522 then talkNpc(p.id) break end
end
```

Copier

map:playersOnMap() → table[]

Autres joueurs visibles : { cell, name, id } par entrée (toi exclu).

map:countPlayers() → number

Nombre de joueurs sur la carte, TOI INCLUS (#map:playersOnMap() ne compte que les autres).

```
if map:countPlayers() > 3 then changeMap("right") end
```

[Copier](#)

```
map:isPlayerOnMap(nom | id) → bool
```

Présence d'un joueur par nom ; par id, teste n'importe quel acteur (monstres compris).

```
map:changeMap(direction) → bool
```

```
map:moveToCell(cell) → bool
```

Bloquant

Compat SnowBot : = changeMap(direction) global (même DSL : "top|right" au hasard, "top(12)", "zaap(id)", "zaapi(id)", "havenbag") et goToCellId(cell). true = arrivé. Le suivi d'équipe fait suivre les membres (pas de SYNC_MOVE).

```
map:changeMap("left|bottom")
map:moveToCell(312)
```

[Copier](#)

```
map:door(cell | "c1|c2") → bool
```

Bloquant**Perm.Gather**

Marche jusqu'à l'interactif de cette cellule (porte, escalier...) et l'utilise, comme la clé d'étape door ; "c1|c2" = une au hasard.

```
map:gather() → bool
```

Bloquant**Perm.Gather**

= gatherAll() : récolte la carte (filtre ELEMENTS_TO_GATHER, ou GATHER).

```
map:fight() → bool
```

```
map:launchFight() → bool
```

```
map:forceFight() → bool
```

Bloquant**Perm.Fight**

fight / launchFight = fight() global (filtres MIN/MAX/FORBIDDEN/FORCE_MONSTERS et MONSTERS_AMOUNT). forceFight() attaque dès qu'un groupe éligible est présent, sinon

ATTEND les repousses sur la carte (parité SnowBot), sans limite de durée : true après le combat (comme fight()), ou tout de suite si déjà en combat, false si la carte a changé, false tout de suite (avertissement) sans la permission « Combat auto ». Pendant l'attente : le bouton Stop, MODERATOR_STOP (popup de modérateur) et les heures PLANNING s'appliquent, et les hooks de paquets / onPrivateMessage sont servis entre deux tranches d'attente.

```
map:ToZaap(mapId [, sourceType]) → bool  
map:toZaap(mapId [, sourceType]) → bool
```

Bloquant

Téléport par le zaap de la carte courante vers mapId (= zaap:go) ; sourceType = 1 → zaapi de la carte (= zaapi:go).

```
map:moveToward(mapId | x, y) → bool  
map:gotomap(mapId | x, y) → bool
```

Bloquant

= travelTo : voyage COMPLET et bloquant jusqu'à la carte. ⚠ SnowBot ne faisait qu'UN changement de carte par appel ; le motif « rappeler à chaque move() » marche quand même.

```
map:loadMove(mapId | x, y | "x,y") → bool  
map:loadMoveToMapId(mapId) → bool  
map:loadMoveToCoord(x, y) → bool  
map:loadRoadToMapId([from,] to) → bool  
map:moveNext() → bool  
map:moveRoadNext() → bool
```

Bloquant

load* mémorise une cible et renvoie true si elle est joignable : à pied par le world-graph, ou via un zaap CONNU qui est la cible ou d'où la cible est joignable à pied (le départ est toujours la carte courante). moveNext / moveRoadNext font le trajet COMPLET jusqu'à cette cible (SnowBot : une transition par appel) ; false sans cible ou déjà arrivé.

```
if map:loadMoveToCoord(-15, 39) then map:moveNext() end
```

Copier

```
map:moveTowardMap(x, y) → bool
map:moveTowardMapId(mapId) → bool
```

Bloquant

UN seul changement de carte dans la direction de la coordonnée cible (au hasard parmi les directions utiles) ; true si déjà sur place.

```
map:listenChange()
map:hasChanged() → bool
```

Pose une marque sur la carte courante / true si la carte (ou ses informations) a changé depuis la marque.

```
map:getNearestZaap(mapId [, exclus]) → number
map:closestzaap(mapId [, exclus]) → number
map:getNearestZaap2(mapId [, exclus]) → number
map:closestzaap2(mapId [, exclus]) → number
map:getNearestZaapbak(mapId [, exclus]) → number
```

Bloquant

Meilleur zaap CONNU pour rejoindre mapId (le moins de cartes à pied ; coût du voyage non vérifié) : mapId du zaap (la cible elle-même si c'est un zaap connu), -1 = le zaap de la carte courante est déjà le meilleur, 0 = aucun. exclus = liste de mapIds à écarter. Liste de zaaps vide : ouvre le zaap de la carte (s'il y en a un) pour l'apprendre, puis le referme. Un zaap trouvé (> 0) charge aussi la cible pour moveNext().

```
-- depuis une carte qui a un zaap
local z = map:getNearestZaap(CIBLE_MAP_ID)
if z > 0 and map:toZaap(z) then map:moveNext()
else travelTo(CIBLE_MAP_ID) end
```

Copier

```
map:knownZaap() → number[]
map:haveThisZaap(mapId) → bool
```

Zaaps connus (liste de mapIds, vide tant qu'aucun dialogue de zaap n'a été vu depuis la connexion) / ce zaap est-il connu ?

`map:monsterGroups() → table[]`

Forme SnowBot des groupes : { cellId, contextualId, monsters = {{ id, level }} } (aussi cell, id, et gid / genericId par monstre), UN élément par monstre, chef du groupe d'abord ; level 0 si inconnu. Voir getMonsters() pour la forme MizanBot.

```
for _, g in ipairs(map:monsterGroups()) do
    print(g.contextualId .. " : " .. #g.monsters .. " monstres en " .. g.cell
end
```

Copier

`map:getArchiMonster() → number[]`

`map:containsArchi() → bool`

genericIds des archimonstres présents / la carte en contient-elle ? map.ArchiMonster (champ, pas une fonction) = liste de tous les genericIds d'archimonstres connus.

`map:players() → table[]`

Joueurs de la carte, TOI INCLUS (forme SnowBot) : { contextualId, id, name, cellId, cell, disposition = { cellId, direction } }.

`map:getMap() → table`

Résumé en lecture seule (SnowBot rendait le message brut de la carte) : { mapId, subArealId, x, y, actors = {{contextualId, cellId, name, kind, npcId}}, monsters = {{contextualId, cellId}}, interactives = {{elementId, cellId, elementTypeId, name}} }. actors[i].npcId = id de modèle d'un PNJ / indice de chasse (0 pour les autres acteurs) ; name = nom seul, comme map:getNpcs().

`map:ForbiddIle(carteZaap, carteCible) → bool`

`map:checkMonsterAmountConstraints() → true`

ForbiddIle (sémantique SnowBot : ce zaap sert-il pour cette destination ?) : true si les deux cartes sont dans la même zone (arealId) ; zones différentes → true, sauf si l'une des deux est une île (nom de zone contenant Île, Saharach, Archipel ou Nimotopia, ou leurs équivalents EN / ES) → false ; zone inconnue → false. checkMonsterAmountConstraints rend toujours true : les contraintes MONSTERS_AMOUNT sont appliquées par le moteur à la sélection du groupe. map.Precompute (champ) vaut false.

Module map: — interactifs & géométrie

Interactifs de la carte (ressources, portes, ateliers, zaaps...), usage fin avec choix du skill, et lectures géométriques pures (aucun paquet envoyé).

```
map: getResources() → table[]
```

Ressources RÉCOLTABLES : { cell, name, skill } par entrée.

```
map: getInteractives() → table[]
```

TOUS les interactifs : { cell, elementId, typeId, elementTypeId, name, action, state, usable, resource, interactionCell, skills = {{uid, id, name, enabled}} } par entrée.

```
map: useById(elementId [, skill]) → ok, outcome
```

```
map: useCell(cell [, skill]) → ok, outcome
```

Bloquant

Perm.Gather

Utilise un interactif (par elementId ou par cellule) et attend l'issue. skill : 0 = défaut, > 0 = uid d'instance puis skillId, -N = N-ième skill actif. outcome : used, ended, mapchanged (ok = true) ; error, timeout, notfound, noskill, noreach, movefail, denied, fight, notsent.

```
local ok, why = map:useCell(312, -1)
if not ok then printWarning("échec : " .. why) end
```

Copier

```
map: cellIdToInteractive(cell) → number
```

```
map: interactiveIdToCell(elementId) → number
```

Conversion cellule ↔ elementId d'un interactif (-1 si absent).

```
map: isWalkable(cell) → bool
```

```
map: isWalkableFight(cell) → bool
```

Case marchable hors combat (relief + obstacles personnalisés) / selon la règle de combat.

```
map: getWalkableCells([hasPath]) → number[]
```

Tableau de numéros de cellules marchables ; hasPath = true → seulement celles atteignables

depuis ta case.

map:**cellDistance**(a, b) → number

Distance entre deux cellules (losange). Utilisable aussi en combat.

map:**adjacentCells**(cell) → table[]

Les 4 voisins { cell } d'une cellule, sans filtre de marchabilité.

map:**getAdjacentCells**(cell) → number[]

map:**getDistance**(a, b) → number

map:**isHandToHand**(a, b) → bool

Noms SnowBot : les 4 voisins en NOMBRES (sans filtre de marchabilité, ≠ adjacentCells qui rend des { cell }), distance = cellDistance, corps à corps = distance ≤ 1.

map:**setCustomObstacles**({c...} | c1, c2, ...) → bool

map:**resetCustomObstacles**()

map:**getCustomObstacles**() → number[]

Obstacles personnalisés évités par le déplacement hors combat : cumulatifs, valables pour la carte courante (vidés au changement de carte).

```
map:setCustomObstacles({ 245, 259 })
```

Copier

map:**sendMapRequest**() → bool

Bloquant

Redemande la carte au serveur et attend la réponse (4 s max) — utile après une désynchronisation.

Module map: — respawn, coffres & maisons à code

Point de respawn (zaap), coffres et maisons protégés par un code. Le code est une CHAÎNE, jamais affichée dans les journaux.

Valeurs de reason

opened, entered, wrong_code, forbidden, busy, code_required, denied, unsupported, timeout, fight, bad_code, not_found, unreachable, owner_not_found.

map:**saveZaap**() → bool

saveZaap() → bool

Bloquant

Sauvegarde le point de respawn sur le zaap de la carte courante (aussi en global).

map:**getSavedZaap**() → number

Carte de respawn connue.

map:**lockedStorage**(cell, code [, skillId]) → ok, reason

Bloquant

Perm.Gather

Ouvre un coffre à code. Le coffre RESTE ouvert : enchaîne putAllItems / depositItem / getAllItems puis leaveDialog.

```
local ok, why = map:lockedStorage(312, "1234")
if ok then putAllItems() leaveDialog() else printWarning("coffre : " .. why)
```

Copier

map:**lockedHouse**(cell, code [, owner [, skillId]]) → ok, reason

Bloquant

Perm.Gather

Entre dans une maison à code ; owner = "Pseudo" ou "Pseudo#1234" quand plusieurs maisons partagent la porte.

map:**isInHouse**() → bool

map:**houses**() → table[]

Suis-je dans une maison ? / maisons de la carte : { houseId, doors = {elementId...}, instances = {{id, owner, tag, locked}} }.

Module combat: — lire le plateau & jouer le tour

État de combat + actions bas niveau pour écrire ta propre IA dans onTurn. Les lectures marchent aussi hors de ton tour (certaines hors combat) ; les actions exigent la permission « Combat auto ».

Fenêtre de tour

castOnCell / moveToCell / endTurn ne sont acceptés que pendant TON tour et sont liés au tour réclamé par onTurn : une fois ce tour fini (retour, budget, fin de combat), ils sont refusés sans envoi. endTurn hors fenêtre est refusé (un fin-de-tour hors tour provoque des tours fantômes).

```
combat:getHp() → number  
combat:getMaxHp() → number  
combat:getPa() → number  
combat:getPm() → number  
combat:getRange() → number
```

PV courants / PV max (en combat : ton combattant ; hors combat : vrais PV) / PA / PM / portée (bonus de PO) du perso.

```
combat:pointsLeft() → pa, pm
```

PA et PM restants pour ce tour.

```
local pa, pm = combat:pointsLeft()
```

Copier

```
combat:getSpellLevel(spellId) → number
```

Niveau appris du sort spellId (0 si non appris).

```
combat:getSpells() → table[]
```

Sorts appris : { id, level } par entrée.

```
combat:getCell() → number
```

Ma cellule dans le combat (-1 si inconnue).

`combat:getTurnNo()` → `number`

Numéro du tour en cours dans ce combat.

`combat:isMyTurn()` → `bool`

true si c'est actuellement mon tour (fenêtre d'action ouverte).

`combat:isPlacement()` → `bool`

`combat:placementCells()` → `table[]`

Phase de placement en cours ? / cases de placement proposées { cell }.

`combat:isAutoFight()` → `bool`

true si le moteur joue les tours de ce bot (profil ou setAutoFight).

`combat:fighters()` → `table[]`

Tous les combattants : { cell, id, enemy, self, hp, hpPercent, lifePoints, maxLifePoints, pa, pm, team, isEnemy, isSummon, isMySummon, alive } par entrée. hp = POURCENTAGE de PV (clé historique) ; lifePoints / maxLifePoints / pa / pm = valeurs absolues (0 si inconnues).

`combat:enemies()` → `table[]`

Ennemis : mêmes clés que fighters() sans enemy/self. Teste e.alive pour ignorer les morts.

```
for _, e in ipairs(combat:enemies()) do
    if e.alive then print(e.id, e.cell, e.lifePoints) end
end
```

Copier

`combat:allies()` → `table[]`

Alliés (toi exclu) : mêmes clés que enemies().

`combat:reachableCells()` → `table[]`

Cellules { cell } atteignables avec tes PM restants.

`combat:fightPath(from, to) → table[]`

Chemin orthogonal { cell } de from à to (from exclu, to inclus) ; PM ignorés, vide si aucun chemin.

`combat:isFreeCell(cell) → bool`

En combat : case marchable sans obstacle ni combattant. Hors combat : marchable et sans acteur.

`combat:hasLos(from, to) → bool`

Ligne de vue entre deux cellules (relief + combattants ; hors combat : relief seul).

`combat:canCast(spellId, cell [, fromCell]) → ok, reason`

Le sort est-il lançable sur cell (depuis fromCell, par défaut ta case) ? reason (codes SnowBot) : 0 OK, 1 trop de lancers, 2 relance, 3 PA, 4 trop de lancers sur la cible, 5 portée max, 6 portée min, 7 pas en ligne, 8 ligne de vue, 10 case libre requise, 11 case occupée requise, 14 pas en diagonale, 15 sort inconnu, 16 inconnu.

```
local ok, why = combat:canCast(161, cible.cell)
```

Copier

`combat:spellZone(spellId, centerCell) → table[]`

Cellules { cell } RÉELLEMENT touchées par le sort spellId lancé sur centerCell (géométrie AoE du sort).

```
local hit = combat:spellZone(161, 305)
```

Copier

`combat:zoneCells(shape, center, minSize, size [, fromCell]) → table[]`

Zone géométrique générique { cell } ; shape = code (88) ou lettre ("X"), orientée depuis fromCell (défaut ta case). Marche hors combat.

combat:castOnCell(spellId, cell) → bool

Bloquant

Perm.Fight

Lance le sort spellId sur la cellule cell (à ton tour). Géométrie et PA sont jugés par le serveur.

combat:moveToCell(cell) → moved, finalCell

Bloquant

Perm.Fight

Se déplace vers cell, tronqué aux PM restants (sinon case atteignable la plus proche sur le chemin), bloquant jusqu'à la confirmation serveur. Renvoie moved, finalCell ; if combat:moveToCell(c) then reste valable.

```
local ok, arrivee = combat:moveToCell(cible.cell)
```

Copier

combat:endTurn() → bool

Perm.Fight

Termine ton tour (une seule fois par tour ; refusé hors de ta fenêtre). Facultatif dans onTurn : la fin de tour est automatique au retour.

combat:playTurn() → bool

Bloquant

Perm.Fight

Dans onTurn : le moteur joue TOUT le tour courant avec le profil de l'onglet Combat (en tenant compte de ce que tu as déjà dépensé) puis le finit.

combat:setAutoFight(bool) → bool

Perm.Fight

Surcharge pour ce run : true = le moteur joue les tours (onTurn n'est plus appelé), false = il ne joue pas. Renvoie l'état effectif.

combat:place(cell) → bool

Perm.Fight

Placement : se met sur une des cases proposées.

`combat:ready([bool=true]) → bool`

Perm.Fight

Placement : prêt / plus prêt (un ready(false) n'est pas écrasé par le prêt automatique).

`combat:kick(id) → bool`

Perm.Fight

Expulse un combattant (pendant le placement ; le serveur décide).

```
-- IA de tour minimale : taper le premier ennemi vivant
function onTurn()
    local cible
    for _, e in ipairs(combat:enemies()) do
        if e.alive then cible = e break end
    end
    if not cible then return end
    if not combat:canCast(161, cible.cell) then
        combat:moveToCell(cible.cell) -- se rapprocher (tronqué aux PM)
    end
    combat:castOnCell(161, cible.cell) -- lancer un sort
end -- fin de tour automatique au retour
```

Copier

Module combat: — réglages, défis, verrous & bilan

Réglages posés par le script en mémoire au-dessus du profil de l'onglet Combat (jamais écrits dans combat.json, effacés au démarrage et à l'arrêt du script), défis, verrous, variantes de sort, abandon et bilan de combat.

`combat:setStyle(v) → bool`

`combat:setTargetMode(v) → bool`

`combat:setSpeed(v) → bool`

Style Aggressive/agressif, Fleeing/fuyard/kite, Passive/passif ; cible Closest/proche, LowestHp/pv, Farthest/loin ; vitesse Instant, Fast/rapide, Slow/lent. false + avertissement si la valeur est inconnue.

```
combat:setStyle("agressif")
combat:setTargetMode("pv")
```

[Copier](#)

```
combat:setKiteRange(min [, max]) → bool
combat:setMaxCasts(n) → bool
combat:setFinishKill(bool) → bool
combat:setSpellEnabled(spellId, bool) → bool
```

Distance de kite 0–20 (max 0 = illimité), plafond de lancers par tour 1–99, achever les cibles, activer/désactiver un sort DÉJÀ présent dans la rotation de l'onglet.

```
combat:setAutoPreFight(bool) → bool
```

Perm.Fight

Placement + « prêt » automatiques.

```
combat:setChallengeMode(mode) → bool
combat:setAutoChallenge(bool) → bool
```

Perm.Fight

"off", "auto" (le moteur valide le 1er défi proposé) ou "script" (ton onPlacement, ou challengeManagement, choisit via challengeProposals / selectChallenge).
setAutoChallenge(true/false) = auto / off.

```
combat:resetProfile() → bool
combat:getProfile() → table|nil
```

Annule tous les réglages du script / profil EFFECTIF { autoFight, autoPreFight, autoChallenge, challengeMode, style, target, speed, finishKill, maxCastsPerTurn, kiteMin, kiteMax, scripted, spells = {{id, priority, enabled, perTurn}} } (nil tant que l'onglet n'a pas poussé de profil).

```
combat:challengeProposals([timeoutMs=3000]) → table[]
```

Bloquant

Placement : demande et attend (0–10000 ms) les défis proposés { id, xpBonus, dropBonus,

state, targets = {{id, cell}} }. Chef d'équipe : attend d'abord les followers (8 s max).

combat:selectChallenge(id) → bool

Bloquant

Perm.Fight

Choisit et valide un défi proposé (chef seulement, une fois par combat).

combat:challenges() → table[]

combat:challengeResult(id) → number

Défis ACTIFS du combat { id, state, status, xpBonus, dropBonus, targets } (status = "running" / "success" / "failed", lisibles après le combat jusqu'au suivant) / résultat d'un défi : 1 réussi, 0 échoué, -1 inconnu.

combat:lock(spec, join, party) → bool

combat:lockState() → table

Perm.Fight

En combat : true = verrouiller l'option (spectateurs / arrivants / groupe seulement), false ou nil = ne pas y toucher ; idempotent, join différé chez le chef tant que les followers n'ont pas rejoint. lockState() = { spec, join, party } confirmé par le serveur. Automatique avec FIGHT_LOCK_*.

combat:setSpellVariant(spellId) → bool

combat:getSpellVariant(spellId) → bool

getSpellVariant(spellId) → bool

Bloquant

Perm.Fight

HORS combat : active la variante spellId (true si elle est déjà active). « get » = obtenir (sémantique SnowBot) ; aussi character:getSpellVariant.

combat:activeVariant(spellId) → number

Membre de la paire de variantes actuellement actif (0 si aucun).

combat:giveUp() → bool

combat:surrender() → bool

Bloquant

Perm.Fight

ABANDONNE le combat courant (= défaite ; l'auto-résurrection prend le relais si besoin). true si le combat est quitté (6 s max).

```
if combat:getHp() < combat:getMaxHp() * 0.1 then combat:giveUp() end
```

Copier

combat:lastFightResult() → table|nil

Bilan du dernier combat : { won, lost, draw, outcome (2 victoire, 1 nul, 0 défaite, -1 inconnu), decoded, durationMs, serverDuration, xp?, kamas?, loot = {{gid, qty, name}}, challenges = {{id, success}}, fightId, msAgo } ; nil tant qu'aucun combat n'a eu lieu depuis le démarrage du client de jeu. decoded = false : won ET lost valent false = issue INCONNUE, pas une défaite (teste r.lost, jamais not r.won). Rend toujours le DERNIER bilan connu, même ancien : utilise msAgo (ms depuis la fin du combat) et/ou fightId (retiens le dernier traité) pour ne pas relire deux fois le même combat.

```
local r = combat:lastFightResult()
if r and r.fightId ~= DERNIER_COMBAT then
    DERNIER_COMBAT = r.fightId
    if r.won then print("Butin : " .. #r.loot .. " lignes")
    elseif r.lost then print("Défaite")
    else print("Issue inconnue") end
end
```

Copier

```
-- Défi choisi par le script + verrous au placement
combat:setChallengeMode("script")
```

Copier

```
function onPlacement()
    combat:lock(true, true, false) -- pas de spectateurs, fermé aux arrivées
    local best
    for _, c in ipairs(combat:challengeProposals(5000)) do
        if not best or c.dropBonus > best.dropBonus then best = c end
    end
    if best then combat:selectChallenge(best.id) end
end
```

Tables fightAction: & fightChallenge: (compat SnowBot)

Alias SnowBot pour porter un script existant. fightAction: renvoie ses listes en TABLEAUX DE NOMBRES (comme SnowBot), pas en { cell }. Le reste de l'IA SnowBot (fightCharacter:, fightBasic:, fightDebug:, fightSlave:) est dans la catégorie suivante.

```
fightAction:castSpellOnCell(spellId, cell) → bool
```

```
fightAction:moveTowardCell(cell) → bool
```

```
fightAction:passTurn() → bool
```

```
fightAction:chooseCell(cell) → bool
```

Perm.Fight

= combat:castOnCell, combat:moveToCell (sans la case finale), combat:endTurn, combat:place.

```
fightAction:canCastSpellOnCell(startCell, spellId, cell) → number
```

Code raison de combat:canCast (0 = OK), ordre d'arguments SnowBot ; startCell nil = ta case.

```
fightAction:inLineOfSight(a, b) → bool
```

```
fightAction:isFreeCell(cell) → bool
```

```
fightAction:getReachableCells() → number[]
```

```
fightAction:getSpellZone(spellId, center) → number[]
```

```
fightAction:getCurrentTurn() → number
```

```
fightAction:getDistance(a, b) → number
```

```
fightAction:isHandToHand(a, b) → bool
```

Lectures : ligne de vue, case libre, cases atteignables, zone d'un sort, n° de tour, distance, corps-à-corps (distance ≤ 1).

```
fightAction:getCells_square(centre, min, rayon) → number[]
```

```
fightAction:getCells_cross(centre, min, rayon) → number[]
```

```
fightAction:getCells_lozenge(centre, min, rayon) → number[]
```

```
fightAction:getCells_diagonal(centre, min, rayon) → number[]
```

```
fightAction:cellsAligned(a, b) → bool
```

```
fightAction:cellsInDiagonal(a, b) → bool
```

Formes de cases (carré, croix, losange, diagonales, du rayon min au rayon max), comme SnowBot : seules les cases MARCHABLES de la carte sont rendues (en combat, relief de

combat ; hors combat, relief), cases occupées par des combattants comprises. Centre gardé quand min = 0 pour la croix, et quand min = 0 et rayon = 0 pour les autres formes. Géométrie de la carte pas encore chargée : forme pure. cellsAligned / cellsInDiagonal : deux cellules alignées / sur une même diagonale.

```
fightAction: GetShortestPath(from, to) → number[]  
fightAction: getCurrentMapId() → number  
fightAction: getNearestEnemy() → number  
fightAction: getNearestAlly() → number  
fightAction: getRealReachableCells() → number[]  
fightAction: isWalkable(cell) → bool  
fightAction: getAdjacentCells(cell) → number[]  
fightAction: getFighter(cell) → number  
fightAction: getEntitiesCount() → number
```

Chemin de combat (from exclu, to inclus), carte courante, CELLULE de l'ennemi / de l'allié vivant le plus proche (-1 si aucun), cases atteignables, case marchable (relief seul : une case occupée par un combattant reste marchable — l'occupation, c'est isFreeCell), 4 voisines, id du combattant vivant sur une case (-1 si aucun ou hors combat), nombre de combattants vivants.

```
fightAction: getAllEntities() → table[]  
fightAction: forceCellidEntity()
```

Combattants VIVANTS, champs SnowBot + MizanBot : Id / id, CellId / cellId / cell, LifePoints / lifePoints, MaxLifePoints / maxLifePoints, AP, MP, Team (sens SnowBot : true = ENNEMI, false = toi ou un allié), isEnemy, Mycharacter, isSummon, Companion (ton invocation), CreatureGenericId et Level (toujours 0 : inconnus). forceCellidEntity est sans effet (l'état suit le serveur).

```
for _, e in ipairs(fightAction:getAllEntities()) do  
    if e.Team then print(e.Id .. " en " .. e.CellId .. " : " .. e.LifePoints  
end
```

Copier

```
fightChallenge: getChallengeIds() → number[]  
fightChallenge: getTargetIds([id]) → number[]  
fightChallenge: getTargetCells([id]) → number[]
```

```
fightChallenge:getXpBonus([id]) → number
fightChallenge:getDropBonus([id]) → number
fightChallenge:getResultFromId(id) → number
fightChallenge:getFightChallenges() → table[]
```

Défis ACTIFS (noms SnowBot challengeAI) ; id absent ou -1 = premier défi. getResultFromId : 1 / 0 / -1 ; getFightChallenges = combat:challenges().

```
fightChallenge:challengeName(id) → string
```

Nom clair d'un défi ("" si inconnu). Aussi quest:challengeName et data:challengeName.

```
fightChallenge:getLastChallengeProposalMessage([timeoutMs=3000]) →
table[]
```

Bloquant

Défis proposés au placement (= combat:challengeProposals) : { id, xpBonus, dropBonus, state, targets }.

```
-- Bloc SnowBot porté tel quel
```

Copier

```
function fightManagement()
  for _, e in ipairs(combat:enemies()) do
    if e.alive and fightAction:canCastSpellOnCell(nil, 161, e.cell) == 0 then
      fightAction:castSpellOnCell(161, e.cell)
      break
    end
  end
  end
  fightAction:passTurn()
end
```

Tables fightCharacter:, fightBasic:, fightDebug: & fightSlave: (compat SnowBot)

Le reste de l'IA SnowBot (characterAI, BasicAI, debugAI) : lectures de ton combattant, rotation de sorts posée par le script puis tour joué par l'IA MizanBot, et journal. À utiliser dans fightManagement() / onTurn. fightSlave: (invocations contrôlées) n'est pas piloté : valeurs neutres.

Rotation du script

fightBasic:clearSpells / addSpell posent une rotation EN MÉMOIRE au-dessus de l'onglet

Combat (jamais écrite dans combat.json, effacée au démarrage et à l'arrêt du script) ; sans clearSpells, addSpell AJOUTE à la rotation de l'onglet telle qu'elle est au moment de l'appel (vide si l'onglet Combat n'a pas encore poussé de profil pour ce perso : appelle clearSpells puis addSpell pour une rotation complète). Le tour est joué par l'IA MizanBot, pas par celle de SnowBot : le résultat peut différer.

```
fightCharacter:getAP() → number
fightCharacter:getMP() → number
fightCharacter:getPO() → number
fightCharacter:getRange() → number
fightCharacter:forceAP(n)
fightCharacter:forceMP(n)
fightCharacter:forcePO(n)
```

PA / PM restants et portée (bonus de PO) de ton perso. force* impose la valeur rendue par get* pour le tour courant de CE combat seulement, jamais reportée sur le même n° de tour d'un combat suivant (utile pour raisonner sur un plan) — rien n'est envoyé au serveur.

```
fightCharacter:getLifePoints() → number
fightCharacter:getLifePointsMax() → number
fightCharacter:getLifePointsP() → number
```

PV, PV max et % de PV de ton perso (0 si inconnus).

```
fightCharacter:getId() → number
fightCharacter:getCellId() → number
fightCharacter:getLevel() → number
fightCharacter:getBreed() → number
fightCharacter:isMule() → bool
fightCharacter:isItMyTurn() → bool
```

characterId, cellule (-1 si inconnue), niveau (0 si inconnu), id de classe, mule = membre d'une équipe MizanBot qui n'en est pas le chef, fenêtre de tour ouverte ?

```
fightBasic:clearSpells()
fightBasic:addSpell(spellId, cible, tours, relances, corpsACorps,
```


pvCible)

clearSpells vide la rotation du script. addSpell ajoute un sort. cible = TeamEnum SnowBot (0–15) : 0 ennemi, 1 allié, 2 soi, 3 / 4 / 5 case libre à côté d'un ennemi / d'un allié / de toi, 6 invocation ennemie, 7 ennemi non invoqué, 8 invocation allié, 9 allié non invoqué, 10 compagnon, 11–15 case libre à côté de 6 / 7 / 8 / 9 / 10. Visent un allié : 1, 4, 8, 9, 10, 13, 14, 15 ; les autres un ennemi (le moteur ne distingue ni invocations ni compagnons). Cibles « case libre » (3, 4, 5, 11–15) : un sort qui exige une case libre (invocation, piège...) est posé sur une case libre à portée (5 : collée à toi) ; tout autre sort vise le combattant lui-même (5 : toi). tours = un lancer tous les N tours ; relances = lancers par tour (1–20) ; corpsACorps = true → portée max 1 ; pvCible = ne lancer que si la cible a au plus ce % de PV (100 = toujours).

```
fightBasic:clearSpells()  
fightBasic:addSpell(SORT_ATTAQUE, 0, 1, 2, false, 100)  -- ennemi, chaque t  
fightBasic:addSpell(SORT_SOIN, 2, 3, 1, false, 50)      -- soi, tous les 3
```

Copier

fightBasic:playTurn(tactique) → bool

Bloquant

Perm.Fight

Dans onTurn / fightManagement (comme combat:playTurn) : le moteur joue TOUT le tour courant avec la rotation effective : 0 passif (immobile), 1 agressif (corps à corps), 2 fuyard (à distance), autre = style inchangé ; 3 = fin de tour sans action. Le style choisi reste posé pour les tours suivants du run.

fightBasic:setApproachDistance(d)

fightBasic:setFightSpeed(v)

setApproachDistance = distance de kite MINIMALE 0–20 (le maximum ne change pas, alors que combat:setKiteRange(d) le remet en plus à illimité) ; setFightSpeed est sans effet (comme chez SnowBot).

fightDebug:print(message)

fightDebug:delay(ms)

Message dans la console (catégorie combat) / pause interruptible (60 s max).

fightSlave:entity() → nil

fightSlave:name() → ""

```
fightSlave:cellId() → -1
fightSlave:canCastSpellOnCell() → -1
fightSlave:getReachableCells() → {}
fightSlave:castSpellOnCell() → false
fightSlave:moveTowardCell() → false
```

Invocations contrôlées NON pilotées : avertissement (une fois) et valeur neutre. entity → nil, name → "" ; index, creatureId, cellId, getNearestEnemy, getNearestAlly, canCastSpellOnCell, canCastSpellOnCellAfterMove → -1 ; getReachableCells, getRealReachableCells, showSpells → {} ; isCellReachable, castSpellOnCell, moveTowardCell → false ; InvalidateCache → nil.

```
-- IA SnowBot « BasicAI » portée telle quelle
```

Copier

```
fightBasic:clearSpells()
fightBasic:addSpell(SORT_ATAQUE, 0, 1, 2, false, 100)
fightBasic:addSpell(SORT_SOIN, 2, 3, 1, false, 50)
fightBasic:setApproachDistance(3)

function fightManagement()
    fightDebug:print("Tour " .. fightAction:getCurrentTurn() .. ", PV " .. fightC
    fightBasic:playTurn(2)    -- 2 = à distance
end
```

Module team: — groupe Dofus & équipe

Groupe Dofus au niveau paquet (invite par nom / accept / leave) + composition de l'équipe MizanBot, ralliement et exécution de code sur les mules. Actions gâtées par la permission « Multi ».

```
team:invite(pseudo) → bool
```

Perm.Multi

Invite un joueur dans le groupe PAR SON NOM.

```
team:accept(partyId) → bool
```

Perm.Multi

Accepte l'invitation de groupe partyId.

```
team:leave(partyId) → bool
```

Perm.Multi

Quitte le groupe partyId.

team:inviteAll() → number

inviteTeam() → number

Perm.Multi

Chef : invite dans le groupe Dofus tous les membres en jeu qui n'y sont pas encore ; renvoie le nombre d'invitations.

team:isInParty([nom | id]) → bool

isInParty([nom | id]) → bool

Sans argument : suis-je dans un groupe ? Avec un nom ou un id : ce joueur est-il dans mon groupe ?

team:partyId() → number

team:partyMembers() → table[]

Id du groupe Dofus (0 si aucun) / membres du groupe { id, name, leader }.

team:members() → table[]

Composition de l'ÉQUIPE MizanBot : { pid, name, leader, mapId, inFight, self } par membre EN JEU (carte/combat VIVANTS).

```
for _, m in ipairs(team:members()) do print(m.name, m.mapId, m.inFight)
```

Copier

team:isLeader() → bool

true si je suis le chef de mon équipe.

team:leaderName() → string

Nom du chef de mon équipe ("" hors équipe).

`team:joinLeader() → bool`

`joinBoss() → bool`

Perm.Multi

Chef : ordonne aux membres absents de sa carte de la rejoindre (non bloquant, = commande /joinBoss).

`team:broadcast(script) → bool`

Perm.Multi

LANCE le snippet Lua donné sur tous les AUTRES membres EN JEU de mon équipe (remplace leur script courant, façon « run on all »).

```
team:broadcast('gatherAll()')
```

Copier

`team:runOnce(nomMembre, code) → bool`

Perm.Multi

Exécute le code Lua UNE fois sur un membre (comme une commande /cmd), sans remplacer son script.

```
team:runOnce("Mule-Bob", 'travelTo(212861955)')
```

Copier

Module inventory:

Lecture de l'inventaire, métadonnées d'objet et équipement (complément des verbes globaux getInventoryItemCount / getKamas / useItem...), plus les noms SnowBot. L'équipement est refusé en combat.

`inventory:items() → table[]`

Tout l'inventaire : { uid, gid, qty, name, pos, equipped } par entrée ; pos = position brute (63 = sac), equipped = (pos ~= 63).

`inventory:equipped() → table[]`

Objets portés (mêmes clés que items()), triés par position.

`inventory:count(gid) → number`

`inventory:itemCount(gid) → number`

Quantité possédée de gid (itemCount = nom SnowBot).

`inventory:category(gid) → string`

`inventory:getTypeName(gid) → string`

`inventory:itemNameId(gid) → string`

Nom de la super-catégorie de gid (ex. "Bois", "Minerai") ; getTypeName = nom SnowBot.

itemNameId(gid) = nom de l'objet (= data:itemName).

`inventory:typeId(gid) → number`

`inventory:itemTypeId(gid) → number`

typeid (catégorie stable) de gid.

`inventory:itemLevel(gid) → number`

`inventory:itemLevelId(gid) → number`

`inventory:getLevel(gid) → number`

`inventory:itemWeight(gid) → number`

Niveau / poids d'un objet (données du jeu).

`inventory:itemPosition(gid) → number`

Position d'un exemplaire : équipé en priorité, 63 = sac, -1 si absent.

`inventory:getPodsPercent() → number`

Pourcentage de remplissage des pods.

`inventory:pods() → number`

`inventory:podMax()` → `number`

`inventory:podP()` → `number`

Pods actuels / max / pourcentage (noms SnowBot). `podP()` rend 100 tant que le poids max est inconnu (garde-fou SnowBot : un bot aveugle part en banque) — `getPodsPercent()` rend alors 0.

`inventory:inventoryContent()` → `table[]`

Forme SnowBot de `items()` : { uid, gid, qty, pos, name, equipped } + `objectUID`, `objectGID`, `quantity`, `position` ; `effects` = {} (effets non exposés).

`inventory:getAveragePrice(gid)` → `number`

Prix moyen d'un objet sur le serveur de jeu de CE bot (table envoyée au client de jeu à la connexion ; partagée entre les bots d'un même serveur, jamais mélangée entre serveurs) ; 0 si inconnu — en mode Socket, la table peut ne jamais arriver.

`inventory:getUID(gid)` → `number`

`inventory:getGID(uid)` → `number`

uid d'un exemplaire de gid (sac d'abord) / gid d'un uid ; -1 si absent.

`inventory:use(gid [, n])` → `bool`

Bloquant

Perm.Gather

Utilise / ouvre n exemplaires (n omis = tous). Alias globaux `useItem` / `useInventoryItem`.

`inventory:useItem(gid [, n=1])` → `bool`

`inventory:useMultipleItem(gid, n)` → `bool`

Bloquant

Perm.Gather

Noms SnowBot. △ `inventory:useItem` utilise UN exemplaire par défaut (comme SnowBot), alors que `inventory:use(gid)` et le global `useItem(gid)` utilisent tout le stack quand n est omis. `useMultipleItem` = n exemplaires.

`inventory:dropItem(gid, n)` → `bool`

Bloquant

Perm.Gather

= dropItem global : jette n exemplaires au sol (piles du sac).

`inventory:openBags()` → `number`

Bloquant

Perm.Gather

Ouvre tous les sacs de ressources ; renvoie le nombre ouvert.

`inventory:delete(gid [, n])` → `bool`

`inventory:deleteItem(gid [, n])` → `bool`

Perm.Gather

DÉTRUIT l'objet (sac seulement, n omis = tout le stack). Alias deleteItem (aussi `inventory:deleteItem`) / `deleteInventoryItem` ; automatique avec `AUTO_DELETE`.

`inventory:equip(gid [, pos])` → `bool`

`inventory:equipItem(gid [, pos])` → `bool`

`inventory:equipUid(uid, pos)` → `bool`

`inventory:equipItemuid(uid, pos)` → `bool`

Bloquant

Perm.Gather

Équipe un exemplaire du sac (pos omis = premier emplacement libre) ou un uid précis à la position pos.

`inventory:unequip(pos)` → `bool`

`inventory:unequipItem(gid)` → `bool`

Bloquant

Perm.Gather

Déséquipe l'emplacement pos (0-15) / l'objet gid porté.

`inventory:stuff([delayMs=500])` → `number`

`inventory:destuff([delayMs=500])` → `number`

Bloquant

Perm.Gather

Remplit les emplacements libres avec l'équipement du sac (niveau ≤ perso, plus haut d'abord,

jamais d'échange) / déséquipe les emplacements 0–15. Renvoie le nombre d'objets bougés ; les emplacements ≥ 16 (monture, compagnon, costume) sont ignorés.

Module bank:

Banque pilotée : le banquier doit être sur la CARTE COURANTE (aucun voyage) ; les dépôts/retraits ciblés exigent un coffre ouvert. Le contenu du coffre est connu après une ouverture de banque.

function bank() et le module bank:

Déclarer fonction bank() ne masque pas le module, avec une nuance de moment. PENDANT le chargement (top-level), une fois fonction bank() déclarée, bank est TA fonction : type(bank) vaut "function" et bank:* échoue à cet endroit (le code placé avant la déclaration voit encore le module). JUSTE APRÈS le chargement, le module est restauré : dans messagesRegistering(), move(), bank(), les hooks et stopped(), type(bank) vaut "table" et bank:open(), bank:withdraw()... fonctionnent. bank() reste callable (métaméthode __call) : ta fonction reçoit alors la table du module en 1er argument (la boucle du moteur l'appelle sans argument). Même règle pour une commande /cmd.

bank:open([npcMapId]) → bool

bank:close() → bool

bank:isOpen() → bool

Bloquant

Perm.Gather

Ouvre la banque (banquier de la carte courante par défaut) / la ferme / est-elle ouverte ?

bank:depositAll([npcMapId]) → bool

Bloquant

Perm.Gather

Ouvre la banque (carte courante par défaut), dépose tout le sac, ferme.

bank:deposit(gid [, n]) → bool

Perm.Gather

Dépose un objet (= depositItem ; n omis = tout le stack).

bank:withdraw(gid [, qty]) → bool

Bloquant**Perm.Gather**

Retire un objet du coffre (qty omis = tout).

`bank:putAllItemsExcept(gids...) → bool`

Bloquant**Perm.Gather**

Dépose tout SAUF ces GID (table {gid...} et/ou liste d'arguments).

`bank:putKamas([n]) → bool`

`bank:withdrawKamas([n]) → bool`

Bloquant**Perm.Gather**

Dépose / retire n kamas (n omis = tout).

`bank:getKamas() → number`

SOLDE en kamas du coffre (-1 si inconnu).

`bank:count(gid) → number`

Quantité de gid dans le coffre.

`bank:items() → table[]`

Contenu du coffre : { uid, gid, qty, name } par entrée.

```
function bank()  
  return {  
    { map = tostring(BANQUE_MAP_ID), custom = function()  -- le bot voyage j  
      if bank:open() then  
        bank:putAllItemsExcept({ 311, 1984 })  -- garde ces objets  
        bank:close()  
      end  
    end },  
  }  
end
```

Copier

Module exchange: (compat SnowBot)

Noms SnowBot. putItem va vers l'établi de craft ouvert, sinon l'échange joueur ouvert, sinon le coffre ouvert ; getItem vers le coffre, sinon l'échange ; les autres verbes d'objets visent le coffre ouvert. Les objets posés viennent des piles du SAC (jamais d'un exemplaire équipé), quantité répartie pile par pile ; un retrait d'un échange joueur ne reprend que ce que tu y as posé.

```
exchange:putItem(gid, n) → bool
exchange:getItem(gid, n [, delay]) → bool
exchange:putAllItems() → bool
exchange:getAllItems() → bool
exchange:putAllItemsExcept(gids...) → bool
exchange:getAllItemsExcept(gids...) → bool
```

Bloquant

Perm.Gather

Pose / retire des objets (voir le routage ci-dessus) en attendant l'accusé adapté avant de rendre la main, comme les accusés SnowBot : un podsP() lu juste après est à jour. putItem : échange joueur = accusé de l'échange ($\leq 1,5$ s) ; coffre = attente que le sac ait perdu toute la quantité, toutes piles (≤ 3 s) ; établi de craft = comme craft:PutItem (une erreur d'échange du serveur n'est pas une pose : true seulement si au moins une pile a vraiment été posée). putAllItems / getAllItems = TOUT le sac / TOUT le coffre : 1re mise à jour (≤ 3 s), puis jusqu'à 300 ms sans nouvelle mise à jour (≤ 3 s de plus) ; retour immédiat s'il n'y a rien à déplacer. getItem dans un échange joueur ne retire que ce qui y est posé ($n \leq 0$ = tout ce qui est posé de ce GID) : false si rien de ce GID n'y est.

```
exchange:putKamas([n]) → bool
exchange:getKamas([n]) → bool
```

Bloquant

Perm.Gather

putKamas : échange = n omis → tous tes kamas ; coffre = dépôt. Δ getKamas(n) RETIRE du coffre (sémantique SnowBot), $\neq$ bank:getKamas() qui lit le solde.

```
exchange:storageItems() → number[]
exchange:storageItemQuantity(gid) → number
exchange:storageItemCount() → number
exchange:storageKamas() → number
```

Lectures du coffre : GID présents, quantité d'un GID, nombre de piles, kamas.

```
exchange:leave() → bool
exchange:ready() → bool
exchange:Ready() → bool
exchange:launchExchangeWithPlayer(id) → bool
exchange:launchExchangeWithPlayerByName(nom) → bool
```

Bloquant

Perm.Gather

leave = bank:close ; ready / Ready = trade:ready(true) ; ouvre un échange avec un joueur par id ou par nom.

Module trade: — échange entre joueurs

Échange avec un joueur VISIBLE sur la carte courante : poser / retirer des objets par GID et des kamas, puis valider. À appeler dans move() / bank(). Gâté par la permission « Récolte auto » (mode mule : « Multi »).

```
trade:withPlayer(nom [, timeoutSec=30]) → bool
```

Bloquant

Perm.Gather

Demande l'échange et ATTEND son ouverture (false si refus ou délai, demande annulée).

```
trade:players() → table[]
```

Joueurs visibles sur la carte : { name, id, cell } par entrée.

```
trade:putItem(gid, n) → bool
trade:removeItem(gid, n) → bool
trade:putKamas(n) → bool
```

Perm.Gather

Pose / retire des objets, pose des kamas dans l'échange ouvert. removeItem ne retire que ce que tu as posé (quantités accusées par le serveur ; $n \leq 0$ = tout ce qui est posé de ce GID) : false si rien de ce GID n'est dans l'échange.

```
trade:ready([bool=true [, timeoutSec=60]]) → bool
trade:validate([timeoutSec=60]) → bool
```